

The Hong Kong Polytechnic University

Department of Computing

COMP4913 Capstone Project

Report (Final)

Interactive Storytelling for Elderly-Friendly RPG Games

Student Name:	Chan Chi Fai
Student ID No.:	22063545D
Programme-Stream Code:	61435-FCS
Supervisor:	LI Amelia
Co-Examiner:	LI Yongqi
2 nd Assessor:	LI Maggie
Submission Date:	10 April 2026

Table of Content

Background and Problem Statement	2
Objectives and Outcome	2
Project Methodology	3
Game scenes	3
Program structure	4
Story generation	4
System design.....	6
Workflow Summary	10
Elderly-friendly function.....	12
Dynamic Font Scaling System	12
The Canvas and Scrollbar mechanism	13
Cantonese Audio Reading (Edge-TTS Integration)	13
Voice Command Input (Speech Recognition, 語音控制)	14
Heartwarming AI Script Generation (Gemini AI Engine).....	15
Time travel(時光倒流) and History(查閱往事).....	15
Simplified save and script Management.....	16
Open step.....	17
Program evaluation mechanism	20
Requirement	24
Possible function and future problem	25

Background and Problem Statement

Nowadays, a lot of games are developed. Different players have different preservatives and needs. But there was no game mainly for the elderly people. This project is going to design an Elderly-Friendly Word Adventure Game. This game will have large fonts, high contrast, simple vocabulary, and calm pacing elements. These elements are good for elderly people, based on their physical condition. The Elderly-Friendly Word Adventure Game will have three layer, Visual, Text and Music. This project focuses on the text layer and aims to design Interactive Storytelling for Elderly-Friendly RPG Games.

Objectives and Outcome

The objective of the project will be to develop story-writing LLM-based agents. The player can input some brief context card (e.g., background, characters), and then the story-writing agents will based on player input provide the scene and choices. The player will use choices to affect the story, based on the choices to provide an ending. Because it is for Elderly-Friendly RPG Games, so project will do different Elderly-Friendly adjustments. This project chose Chinese to do the game language. Because I am in Hong Kong, before level was lower compared to nowadays. The elderly have less chance to get a very good educational environment for learning English. So using Chinese to be the game language can allow more elderly people in Hong Kong can play.

Project Methodology

This project will use Python 3.12 be the basic and the LLM API to do the scene text and choices generation. This project will use the model gemini-flash-latest to generate the story. Because Gemini has a free quote and gemini-flash-latest will use the least api quote. It is suitable for the

development stage, because this stage has a lot of bugs that need to be fixed. Maybe can use another advanced model when the system is more complete,

```
API_KEY = ""
client = genai.Client(api_key=API_KEY)
MODEL_NAME = "gemini-flash-latest"
SCENES_FILE = "scenes.json"
BACKUP_DIR = "backups"
```

Game scenes

And then is about the game scenes. This project will be planned to be categorized as static or dynamic scenes. Static scenes are predefined scenes stored in JSON files. The static scenes are those scenes where the player saves data in the previous game. Dynamic scenes are generated by an LLM based on player input. This project chose JSON file to be the scenes structure because the JSON file is highly readable and flexible. JSON files have a clear structure, making them suitable for storing scenario descriptions and options. JSON files can be modified as needed to add more scenarios or options. Another possible structure is SQLite or CSV. SQLite is suitable for storing large amounts of scene data, supporting queries and indexing. But SQLite configuration and usage are relatively complex, with a high learning curve, and may be over-engineered for small games. CSV is simple but it difficult to store complex scenarios and option data, and lacks the flexibility of JSON.

```

{
  "scenes": {
    "scene_1": {
      "description": "陳日站在古老王城的廢墟前，手中緊握著一枚公主遺落的髮簪，那是他踏上征途的唯一信物。風帶來了魔王城堡方向陰冷的氣息，但他心中燃燒的是對正義的堅",
      "options": [
        {
          "text": "即刻啟程，向著魔王城下的低語森林前進。",
          "next_scene": "scene_2"
        }
      ]
    },
    "scene_2": {
      "description": "低語森林的入口被巨大的藤蔓和濃密的黑暗籠罩，連陽光都難以穿透。陳日深吸一口氣，知道這片森林充滿了幻覺與陷阱。他必須找到一條既快速又安全的道路",
      "options": [
        {
          "text": "選項A: 憑藉自己的力量和斧頭直接砍出一條最短的直線路徑。",
          "next_scene": "scene_3"
        },
        {
          "text": "選項B: 輕聲呼喚森林裡的小精靈，希望能得到他們友善的指引。",
          "next_scene": "scene_4"
        }
      ]
    },
    "scene_3": {
      "description": "陳日決定依靠自己的雙手。他揮舞斧頭，汗水浸濕了衣襟，每一斧都充滿力量。雖然開闢了道路，但巨大的動靜驚動了森林深處一群脾氣暴躁的木靈。他必須加",
      "options": [

```

Program structure

For the program structure, this project will use a dictionary structure to manage scene logic. The dictionary structure is simple and direct. It allows for a quick lookup of the current scene and options. And the dictionary structure is a clear structure. Nested dictionaries allow for direct expression of scene descriptions, options, and the logical relationships between scenes. And it directly corresponds to the JSON file structure. It is easier to apply the scene management function.

Story generation

To generate the dynamic game scene and options, this project tries two ways to achieve this. One is to generate one scene and scene options, then let the player choose one option and then generate the next scene. But the generated scene has a logic problem or some information will appear that did not appear before in the next scene. It reduces the story's completeness.

```

"Forest_Rabbit_Help_3A": {
  "description": "小一來彩虹樹林。樹葉是軟軟的棉花糖。地上有好多亮晶晶的石頭。可是路燈壞掉了。這裡變得很黑。我要",
  "options": [
    {
      "text": "抬頭看，花朵是不是掛在樹上？",
      "next_scene": "S3_LookUp"
    },
    {
      "text": "彎腰找，花朵是不是躲在草叢裡？",
      "next_scene": "S3_LookDown"
    }
  ]
},
"S3_LookUp": {
  "description": "天黑了。小一肚子餓扁了。他找到一片閃亮的樹叢。樹上結滿了紅色的果實。果實暖暖的，像小燈籠。吃下",
  "options": [
    {
      "text": "先睡一下，抱著果實。",
      "next_scene": "scene_5_rest"
    },
    {
      "text": "趕快爬上最高的塔。",
      "next_scene": "scene_5_climb"
    }
  ]
}

```

Another method is to ask the LLM to generate a whole story and options. It increases the story's completeness, the scene becomes more logical. The scene can match the previous scene. For a story, completeness and logic are very important. So this project chose to let LLM generate the whole story at the beginning.

```

"scene_8": {
  "description": "魔王城堡高聳在黑色的山脊上，散發著令人不安的氣息。城堡周圍佈滿了高階的黑暗魔法結界，肉眼可見的紫色光芒閃爍。想要進入，必須",
  "options": [
    {
      "text": "選項A: 尋找結界的能量源頭，試圖用物理力量摧毀它。",
      "next_scene": "scene_9"
    },
    {
      "text": "選項B: 在城堡周邊巡視，尋找是否有隱藏的僕人通道或未被結界覆蓋的後門。",
      "next_scene": "scene_10"
    }
  ]
},
"scene_9": {
  "description": "陳日注意到結界的力量似乎來自城堡頂部的一塊黑色晶石。他決定繞著外牆向上攀爬，這是一項極其危險的任務，隨時可能被守衛發現或被",
  "options": [
    {
      "text": "開始向上攀爬，目標是黑色晶石。",
      "next_scene": "scene_11"
    }
  ]
}

```

After LLM generates the story, let LLM return the story in a specific format and store it in the scenes.json file. And then use GUI to show the story scene and the option.

```

格式: {{ "scenes": {{ "scene_1": {{ "description": "...", "options": [ {{ "text": "...", "next_scene": "..." }} ] }} }} }}
"""

```

```

} scenes.json > ...
1  {
2    "scenes": {
3      "scene_1": {
4        "description": "陳日站在古老王城的廢墟前，手中緊握著一枚公主遺落的髮簪，那是他踏上征途的唯一信物。",
5        "options": [
6          {
7            "text": "即刻啟程，向著魔王城下的低語森林前進。",
8            "next_scene": "scene_2"
9          }
10       ]
11     },
12     "scene_2": {
13       "description": "低語森林的入口被巨大的藤蔓和濃密的黑暗籠罩，連陽光都難以穿透。陳日深吸一口氣，知道這將是一場艱難的旅程。",
14       "options": [
15         {
16           "text": "選項A：憑藉自己的力量和斧頭，直接砍出一條最短的直線路徑。",
17           "next_scene": "scene_3"
18         },
19         {
20           "text": "選項B：輕聲呼喚森林裡的小精靈，希望能得到他們友善的指引。",
21           "next_scene": "scene_4"
22         }
23       ]
24     }
25   }
26 }

```

System design

The system is divided into five main logical layers.

Environment Stability Layer: Handles encoding issues, ensuring the correct display of Chinese characters (UTF-8) across different terminals. In the development process, Chinese characters are one of the bugs that will appear most often. So use this layer to fix the encoding issues.

```

# --- 1.Environment Stability Layer---
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')
sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8', errors='replace')

```

Configuration Management Layer: Centrally manages API keys, model names, and file paths.

```

# --- 2. Configuration Management Layer ---
API_KEY = ""
client = genai.Client(api_key=API_KEY)
MODEL_NAME = "gemini-flash-latest"
SCENES_FILE = "scenes.json"
BACKUP_DIR = "backups"

```

Data Handling Layer: Responsible for reading and writing JSON files and automatic backup (Backup System). This is crucial for games for the elderly. The elderly may not be familiar with computers or appear to have memory decline issues, so preventing the loss of old memories when generating new stories.

```
# --- 3. Data Handling Layer ---
def save_json(file_path, data):
    try:
        if not os.path.exists(BACKUP_DIR): os.makedirs(BACKUP_DIR)
        if os.path.exists(file_path):
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            os.rename(file_path, os.path.join(BACKUP_DIR, f"backup_{timestamp}.json"))
        with open(file_path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=4)
    except: pass

def load_json(file_path, default):
    if not os.path.exists(file_path): return default
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            data = json.load(f)
            return data if isinstance(data, dict) else default
    except: return default
```

AI Generation Engine: This is the heart of the system. Through carefully designed Prompt Engineering, the AI needs to follow the player input and the script structure and logical norms to form the story. And the structured JSON output is mandatory.

Prompt Engineering Design:

Scale Limitation: A mandatory requirement of 20 scenes ensures sufficient length and depth for the stories.

Causal Continuity: Options must be "solutions to problems," aligning with RPG selection logic. To increase the story effect of those options. Let the player think they are affecting the story to increase their story engagement.

Safety and Style: Double quotes are disabled to prevent JSON parsing crashes, and a gentle,

conversational writing style is specified to suit older users.

Structural Integrity: Scenes from scene_1 to scene_20 must be consecutive to increase the story completeness.

Positive ending: Make the ending must be simple, positive, hopeful, and heartwarming, reflecting the virtues of good deeds being rewarded or hard work being rewarded.

```
# --- 4. AI Generation Engine ---
def generate_full_story(config):

    prompt = f"""
    你是一位暖心故事作家。請生成一個包含 20 個場景的 JSON 網狀敘事劇本。

    【核心目標】：
    將 20 個場景根據結局數量分配為幾條完整、連續的故事長鏈。

    【故事設定】：
    - 角色：{config['name']} ({config['personality']})
    - 目標：{config['goal']} | 世界：{config['world']}

    【劇本結構與邏輯規範】：
    1. **正向簡單結局**：必須且只能生成 2 到 3 個結局場景（如 scene_18, 19, 20）。結局必須是簡單、正向、充滿希望且溫馨的，體現善有善報或努力有成的美德。
    2. **路徑分配機制**：
        - 根據結局數量，將大部分場景（約 15-17 個）用於構建完整的故事主線。
        - 分歧點與選項必須設計為：『解決當前事件或問題的不同方法』。
        - 例如：遇到斷掉的小橋，選項 A 是『動手修復』，選項 B 是『向鄰居求助』。不同的方法會引導至不同的因果場景，但最終都要組成連貫的故事。
    3. **因果連續性**：嚴格遵守上下文邏輯。場景描述必須明確回應玩家選擇的方法所產生的直接影響。
    4. **禁止劇情穿越**：嚴禁提及玩家未經歷的路徑事實。
    5. **分段檢查**：要求模型在生成時，內部自我檢查場景編號是否連續。
    6. **JSON 完整性**：明確規定 scene_1 到 scene_20 必須全部出現，缺一不可。

    【寫作風格】：
    - 每段 80-130 字，溫柔且富有畫面感的中文口語。
    - 禁止使用雙引號（"），請用單引號（'）。
    - **嚴禁回傳 Markdown 標籤**，只回傳純 JSON。

    格式：{{ "scenes": [ {{ "scene_1": {{ "description": "...", "options": [ {{ "text": "...", "next_scene": "..." }} ] }} ] }} }}
    """
```

```
try:
    response = client.models.generate_content(model=MODEL_NAME, contents=prompt)
    raw_text = response.text.strip()

    # Parsing Enhancement: Ensure accurate JSON extraction during long postbacks.
    clean_text = raw_text.replace("```json", "").replace("```", "").strip()
    start = clean_text.find('{')
    end = clean_text.rfind('}') + 1

    if start == -1 or end == 0: return None

    result = json.loads(clean_text[start:end])
    if isinstance(result, list): result = result[0]
    return result if isinstance(result, dict) and "scenes" in result else None
except:
    return None
```

GUI Interaction Layer: Based on tkinter category design, manages view switching (main menu,

settings page, game page).

```
class AdventureGame:
    def __init__(self, root):
        self.root = root
        self.root.title("精彩人生冒險")
        self.root.geometry("1400x1000")
        self.root.configure(bg=█ "#FDFCF0")

        self.fonts = {
            "title": ("Microsoft JhengHei", 64, "bold"),
            "text": ("Microsoft JhengHei", 36),
            "btn": ("Microsoft JhengHei", 30, "bold")
        }

        self.story_data = {}
        self.current_scene_id = "scene_1"
        self.show_main_menu()

    def show_main_menu(self):
        for w in self.root.winfo_children(): w.destroy()
        tk.Label(self.root, text="精彩人生冒險", font=self.fonts["title"], bg=█ "#FDFCF0", fg=█ "#4A4A4A").pack(pady=80)
        tk.Button(self.root, text="開始故事", font=self.fonts["btn"], width=30, height=2, bg=█ "#FFF9C4", command=self.setup_story).pack(pady=10)
        tk.Button(self.root, text="讀取舊故事", font=self.fonts["btn"], width=30, height=2, command=self.load_last_story).pack(pady=10)

    def setup_story(self):
        setup_win = tk.Toplevel(self.root)
        setup_win.title("故事背景設定")
        setup_win.geometry("1300x1000")
        setup_win.configure(bg=█ "#F0F4C3")

        fields = [
            ("世界背景", "world", "西方奇幻世界"),
            ("角色名字", "name", "陳日"),
            ("性格描述", "personality", "勇敢且熱心腸"),
            ("角色目標", "goal", "拯救被魔王囚禁的公主")
        ]

        entries = {}
        for label_text, key_name, hint in fields:
            tk.Label(setup_win, text=label_text, font=("Microsoft JhengHei", 24), bg=█ "#F0F4C3").pack(pady=5)
            ent = tk.Entry(setup_win, font=("Microsoft JhengHei", 28), width=60)
            ent.insert(0, hint)
            ent.pack(pady=5)
            entries[key_name] = ent

        def confirm_start():
            cfg = {k: v.get() for k, v in entries.items()}
            setup_win.destroy()
            self.create_story(cfg)

        tk.Button(setup_win, text="生成劇本", font=self.fonts["btn"], bg=█ "#8BC34A", fg="white", command=confirm_start).pack(pady=30)

    def create_story(self, config):
        for w in self.root.winfo_children(): w.destroy()
        tk.Label(self.root, text="正在為您撰寫劇本...\n舊檔案已安全備份。", font=self.fonts["text"], bg=█ "#FDFCF0").pack(pady=150)
        self.root.update()

        new_data = generate_full_story(config)
        if new_data:
            self.story_data = new_data
            save_json(SCENES_FILE, self.story_data)
            self.current_scene_id = "scene_1"
            self.show_scene()
        else:
            messagebox.showwarning("提示", "20個場景生成量較大，AI解析失敗。請點擊確定再試一次。")
            self.show_main_menu()
```

```

def show_scene(self):
    for w in self.root.winfo_children(): w.destroy()
    scene = self.story_data.get("scenes", {}).get(self.current_scene_id)

    if not scene:
        self.show_main_menu()
        return

    tk.Label(self.root, text=f"↑ 當前位置: {self.current_scene_id}", font=("Arial", 10), bg="■ #FDFCF0", fg="gray").pack(anchor="ne", padx=20, pady=20)

    # description
    tk.Label(self.root, text=scene.get("description", ""), font=self.fonts["text"],
            bg="■ #FDFCF0", wraplength=1000, justify="left", padx=50, pady=30).pack()

    # options
    options = scene.get("options", [])
    if options and isinstance(options, list):
        for opt in options:
            tk.Button(self.root, text=opt.get("text", "探索下一步"), font=self.fonts["btn"], bg="■ #E1F5FE", height=2,
                    command=lambda s=opt.get("next_scene"): self.go_to(s)).pack(pady=8, padx=100, fill="x")
    else:
        tk.Label(self.root, text="🏠 旅程圓滿完結 🏠", font=self.fonts["btn"], fg="■ #4CAF50", bg="■ #FDFCF0").pack(pady=30)
        tk.Button(self.root, text="回主選單", font=self.fonts["btn"], command=self.show_main_menu).pack(pady=10)

```

```

def go_to(self, next_id):
    if next_id in self.story_data.get("scenes", {}):
        self.current_scene_id = next_id
        self.show_scene()
    else:
        self.show_main_menu()

def load_last_story(self):
    if os.path.exists(SCENES_FILE):
        data = load_json(SCENES_FILE, {})
        if "scenes" in data:
            self.story_data = data
            self.current_scene_id = "scene_1"
            self.show_scene()
        return
    messagebox.showwarning("提示", "目前沒有有效的存檔。")

if __name__ == "__main__":
    root = tk.Tk()
    AdventureGame(root)
    root.mainloop()

```

Workflow Summary

1. Initialization Phase: The program starts and initializes the Tkinter window.

The main menu is displayed, allowing players to choose "Start Story" or "Load Old Story".

2. Setup & Request Phase:

User Input: Players input the world background, character name, personality, and goals in the interface.

Prompt Encapsulation: The system incorporates these variables into a preset prompt template.

API Call: The system simultaneously sends a request to Google Gemini.

3. Generation & Parsing Phase:

Story Generation: Gemini generates JSON strings conforming to the specifications of 20 scenes.

Data Cleaning: The program receives the strings, removes noise, and converts them into a Python dictionary.

Archiving and Backup: The successfully parsed dictionary is written to scenes.json, and a timestamp backup of the old file is performed.

4. Game Loop Phase

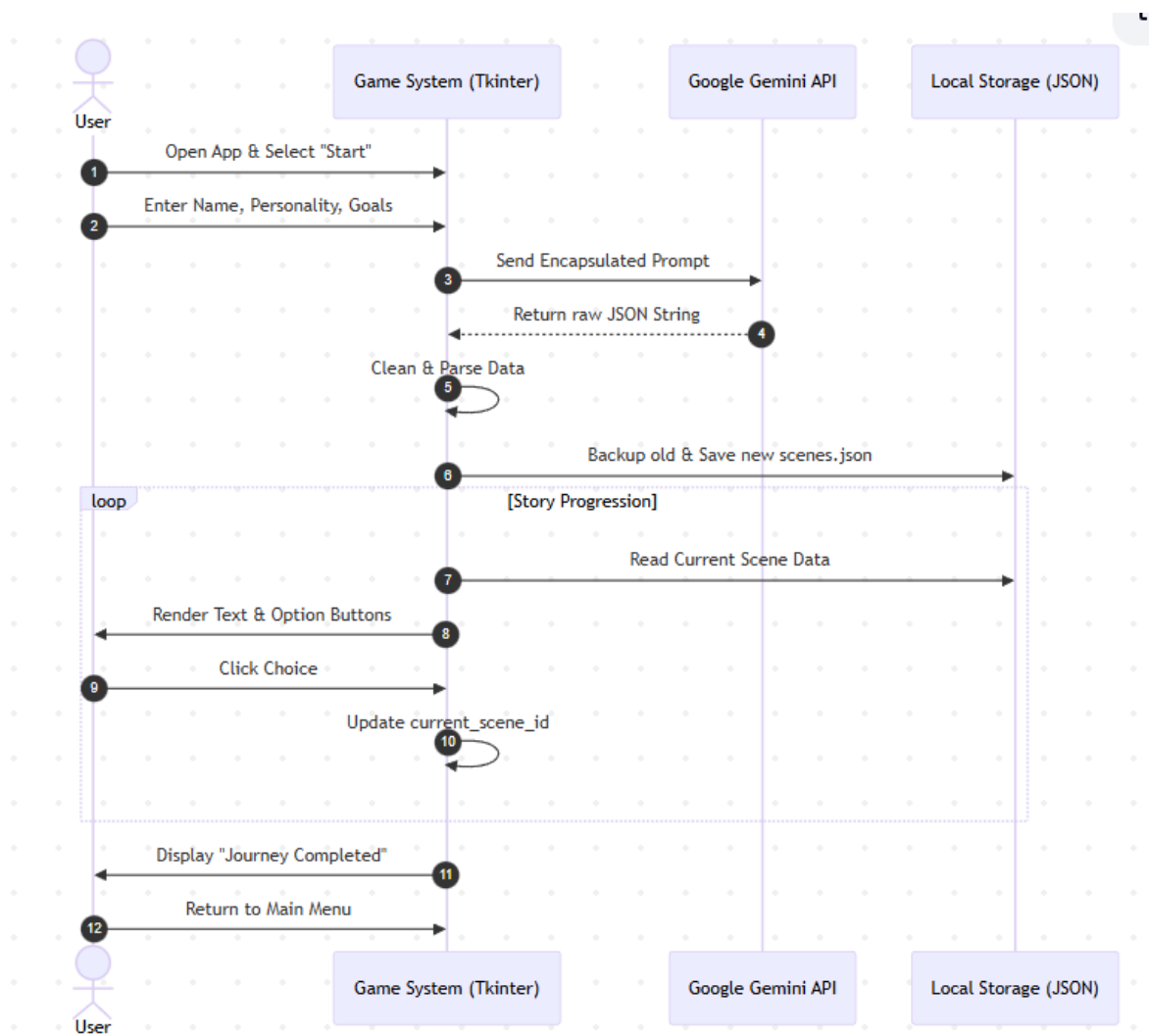
Loading the first scene: Starts from scene_1 by default.

View Rendering: Displays the scene description text on the screen and dynamically generates option buttons.

State Transition: After the player clicks a button, the program reads the next_scene ID of that option, jumps to the corresponding scene, and repeats this loop.

5. Ending Phase

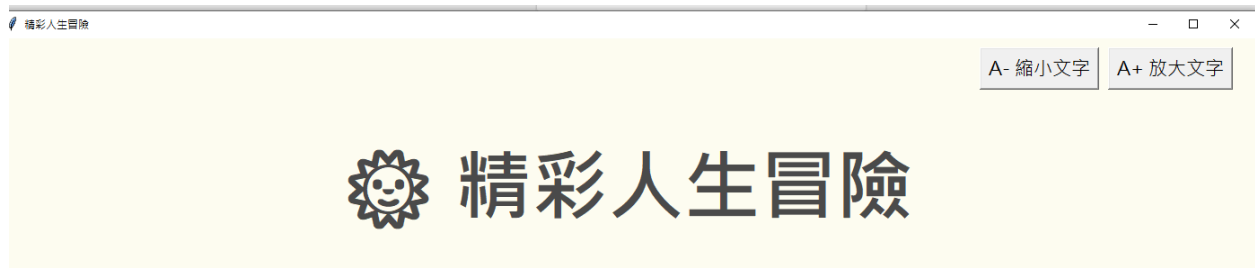
When no options are detected in the scene, the interface determines it as the ending, displays "Journey Completed" and provides a button to return to the main menu.



Elderly-friendly function

The target people of this project are the elderly. So the project has achieved a lot of elderly-friendly functions.

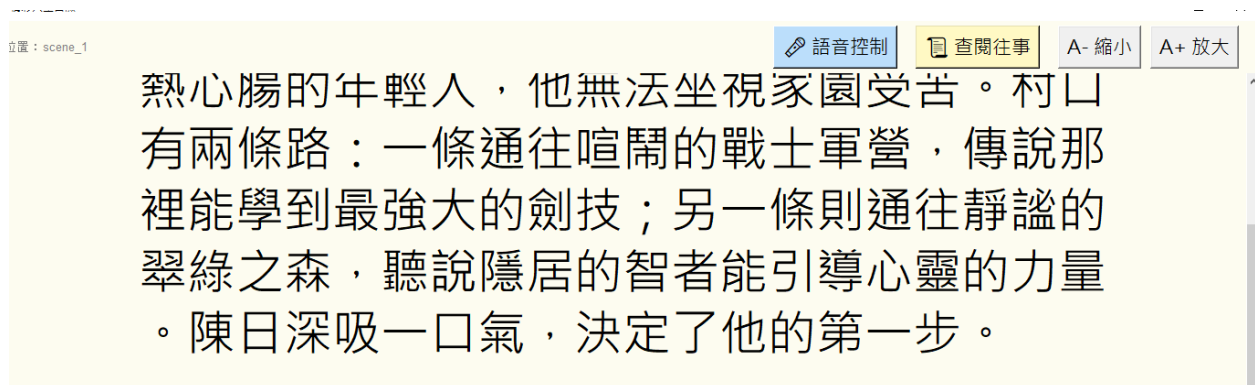
Dynamic Font Scaling System



How it works: The global font dictionary (self.fonts) is dynamically modified via the `change\_font\_size` method and the interface is refreshed in real time.

Why it is elderly-friendly: It solves common presbyopia or vision deterioration problems among the elderly. The elderly can adjust the font to the most comfortable size by clicking the "A+" or "A-" buttons. Because different elderly their vision have different level down, it improves reading autonomy for all elderly users.

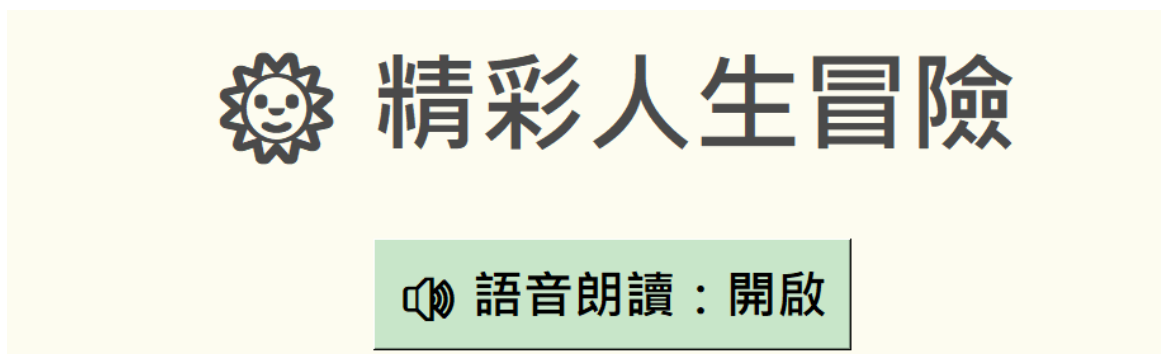
The Canvas and Scrollbar mechanism



For match font scaling system, to avoid the text overlapping the windows. The Canvas and Scrollbar mechanism allows users to view the full content via scrolling or the mouse wheel, regardless of the font size.

Scroll wheel support: Added mouse wheel event binding for more intuitive operation.

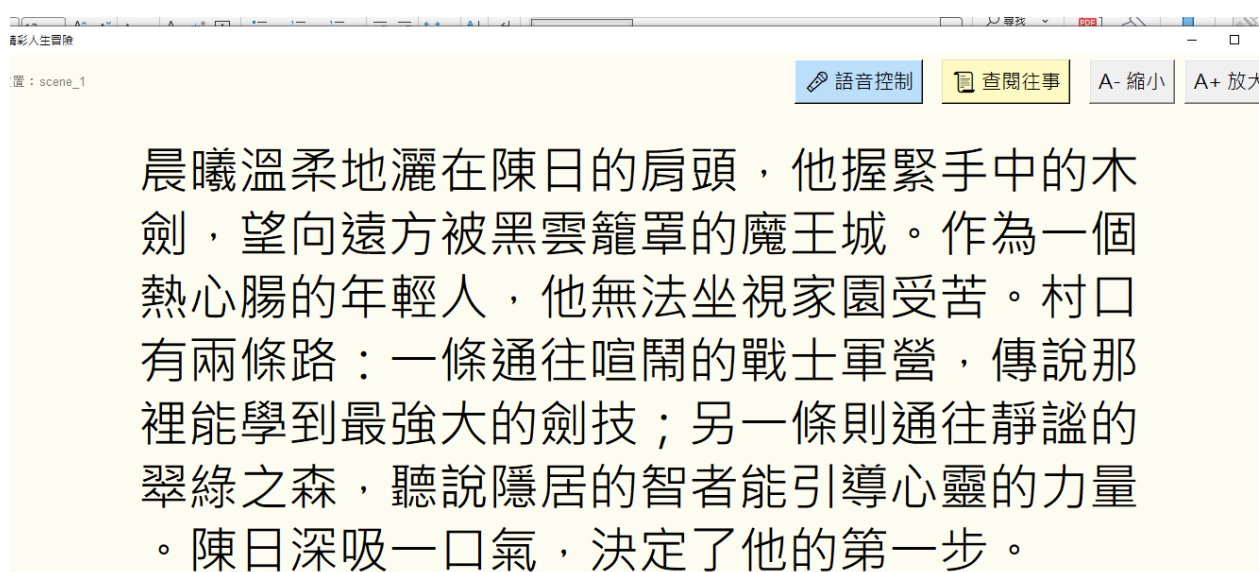
Cantonese Audio Reading (Edge-TTS Integration)



How it works: The edge-tts engine converts the story text into audio, which is then played via pygame. Based on our game target user is the Hong Kong local user. A Hong Kong female voice (zh-HK-HiuGaiNeural) was specifically chosen.

Why it's elderly-friendly: Considering that some elderly people may find reading long texts difficult or prefer listening to stories. Also, elderly vision may be damaged. Audio makes the stories more engaging. The autoplay function reduces the time elderly people spend staring at the screen, alleviating eye strain.

Voice Command Input (Speech Recognition, 語音控制)



How it works: After clicking "Voice Control" bottom, the program listens to the microphone.

Converts speech to text using speech recognition and automatically matches options (such as saying "第一項" or "第二項"). Since the target users are Hong Kong users, we use Cantonese as the input language.

Why it's elderly-friendly: Designed for seniors with arthritis or hand tremors who have difficulty accurately operating a mouse. Enables "hands-free" operation. Seniors simply speak the item they want to select and the system automatically redirects. Lowering the barrier to interaction with the computer. Also, the user can choose to open or not.

Heartwarming AI Script Generation (Gemini AI Engine)

How it works: Generates a network narrative of 20 scenes using the Gemini API. Prompts emphasize a "heartwarming writer" style and a "positive ending".

Why it's heartwarming: Emotional support is crucial for seniors. Ensures the story is gentle, non-oppressive and logically simple. This provides cognitive stimulation for seniors while offering a sense of psychological companionship.

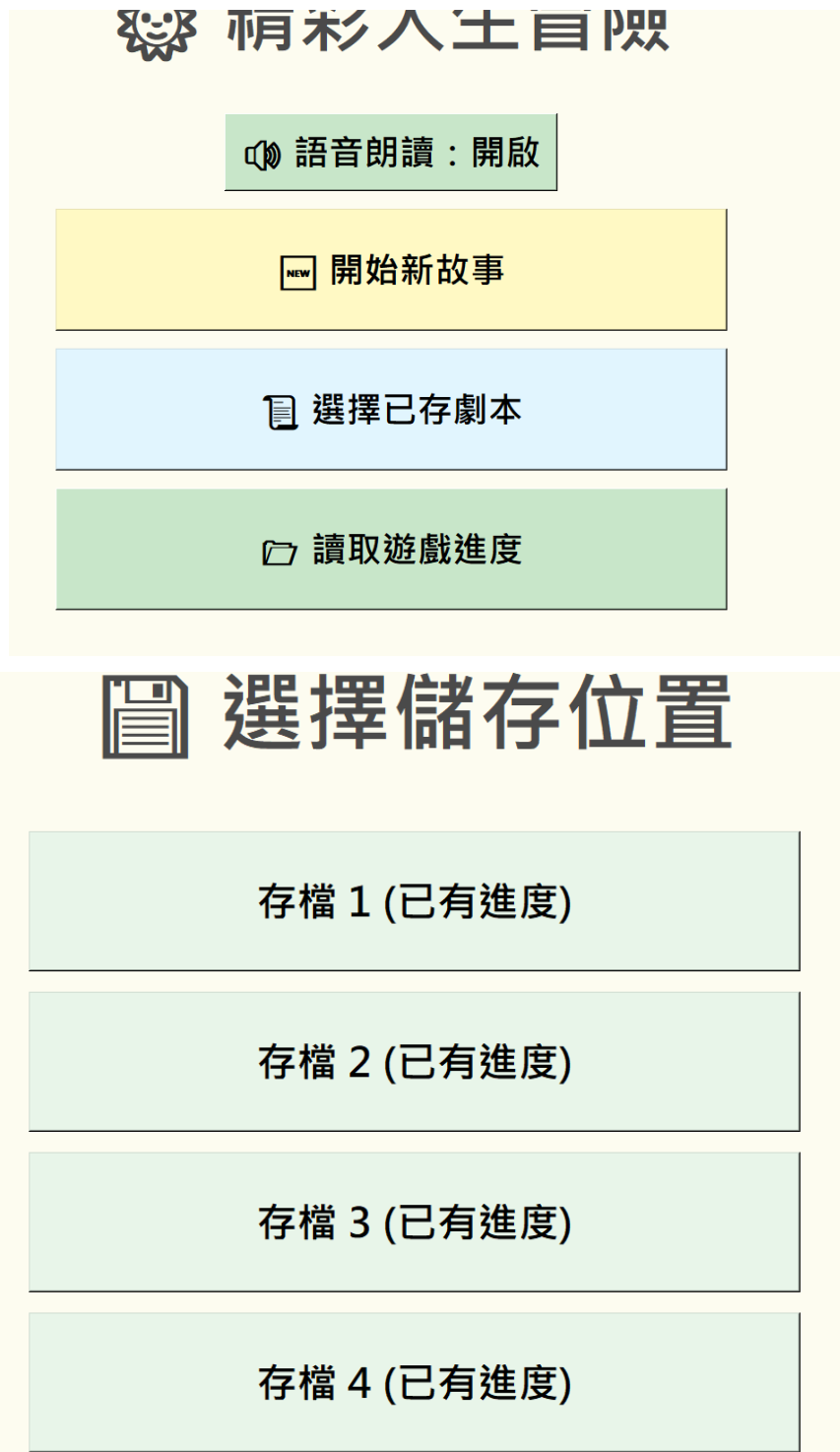
Time travel(時光倒流) and History(查閱往事)



How it works: `undo\_move` allows you to go back to the previous step. `show\_history` pops up a scrolling window displaying all past scene descriptions and choices.

Why it's elderly-friendly: To alleviate anxiety caused by short-term memory decline. In case older players forget previous events or accidentally select the wrong option, they can "查閱往事" or "時光倒流" at any time, making the game stress-free and error-free.

Simplified save and script Management



How it works: Provides 4 visual save slots, automatically labeled "Progress Already"(已有進度) or "Empty"(空).

Why it's user-friendly: Simplifies the complex file management logic of traditional computers.

Seniors don't need to deal with file paths or enter file names; simply click a prominent button (such as "存檔 1") to save, making it easy to continue playing next time.

Open step

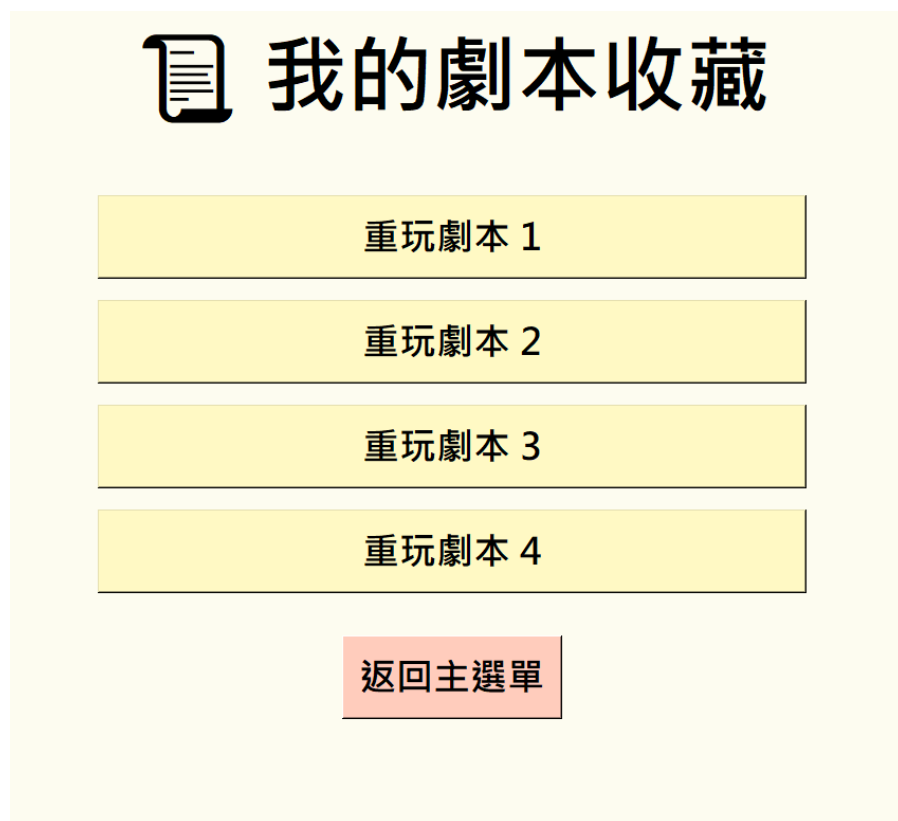


Choose an open new story or an old story.

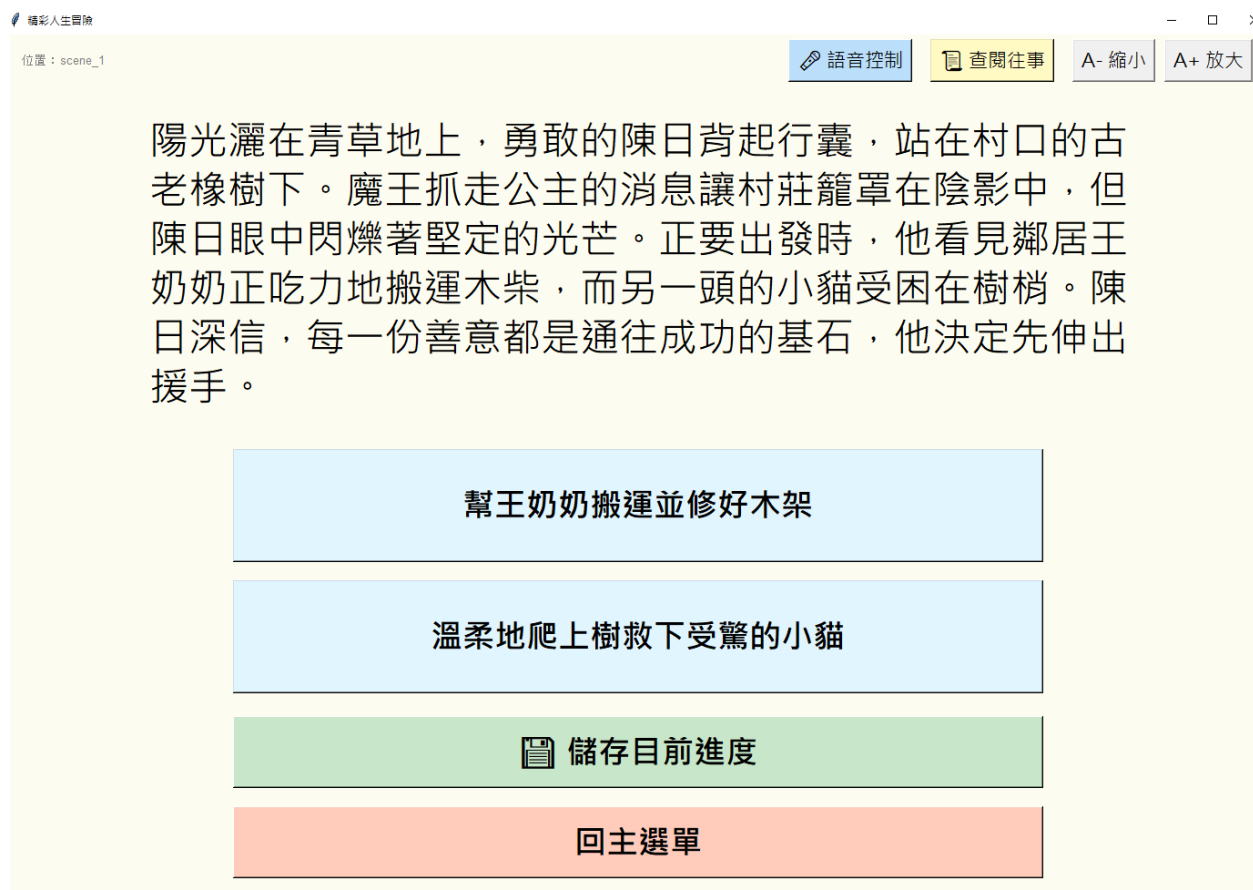
New story



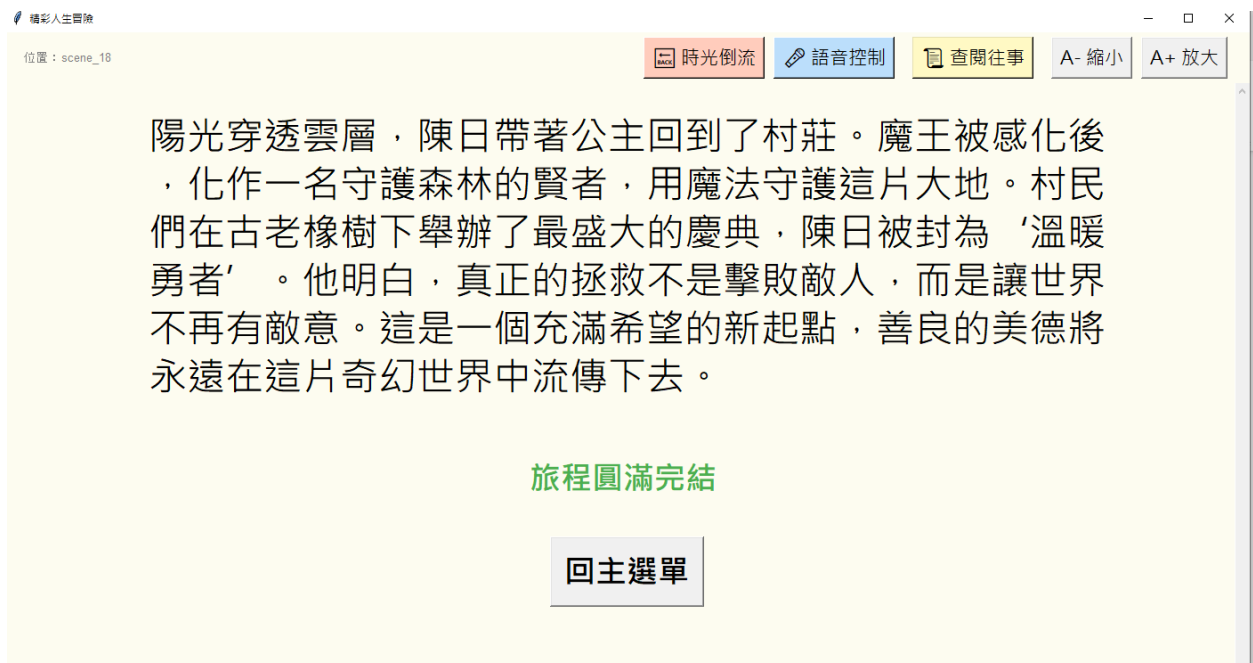
Old story



Game process



End



Program evaluation mechanism

To rigorously assess my system, I establish a "gold standard" dataset as a standard for comparison. Using advanced LLMs GPT-4 and Qwen to draft these "gold standard" responses based on specific input, followed by my verification to ensure quality. Treat this process like designing an exam: create specific questions (inputs) and an answer key (ground truth) to objectively test my system.

For testing the system's ability, I manually set a lot of different characters, background world and characters' goals. Those inputs cover the nowadays most common and popular story setting. And input to ask GPT-4 and Qwen to follow those inputs to output the whole story and output it into a .jsonl file. Use to file to be my dataset and write a program eval_story.py to call my system to create the story to do the compression.

Here are those specific inputs:

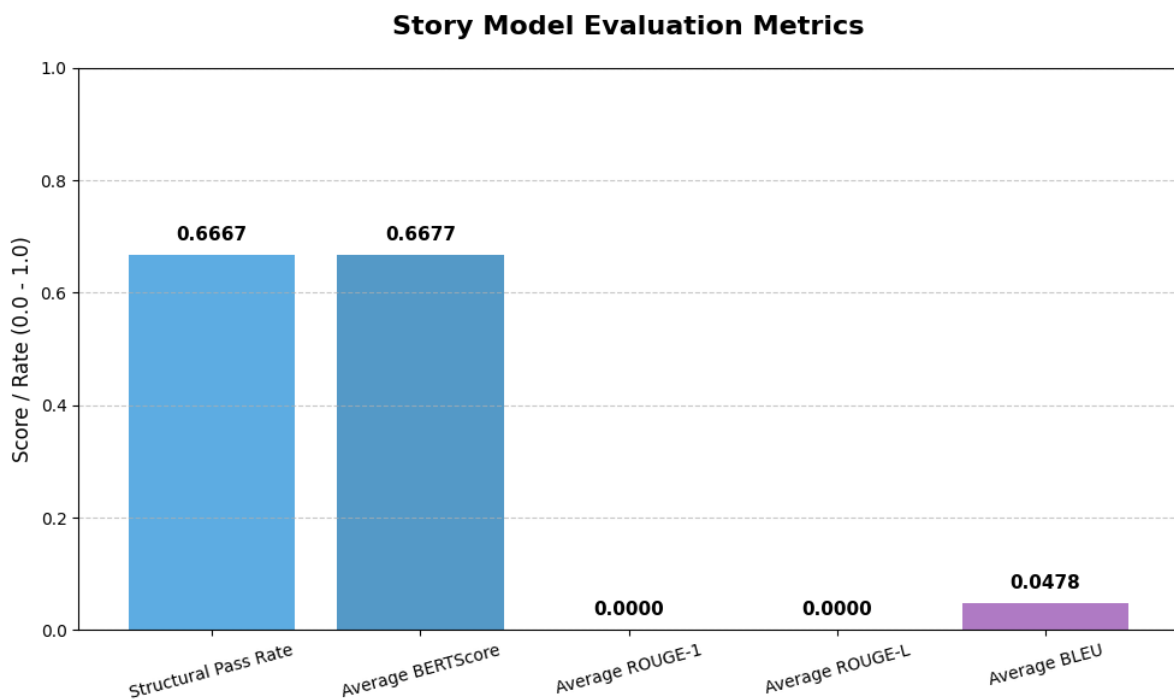
```

1  {"input": {"name": "星海", "personality": "樂觀豁達", "world": "星際殖民艦", "goal": "尋找適合居住的新地球"}, "gold_output": {"scenes": {"scene_1": {"des
2  {"input": {"name": "艾莉絲", "personality": "充滿正義感", "world": "蒸汽龐克都市", "goal": "揭開工廠污染真相"}, "gold_output": {"scenes": {"scene_1": {"d
3  {"input": {"name": "葉問", "personality": "低調內斂", "world": "近代佛山街道", "goal": "守護鄰里安全"}, "gold_output": {"scenes": {"scene_1": {"descript
4  {"input": {"name": "林克", "personality": "好奇心旺盛", "world": "神秘地底王國", "goal": "尋找永恆之光"}, "gold_output": {"scenes": {"scene_1": {"descri
5  {"input": {"name": "白婆婆", "personality": "慈祥且富有耐心", "world": "童話森林", "goal": "為迷路的小動物們舉辦慶典"}, "gold_output": {"scenes": {"scene
6  {"input": {"name": "莫克", "personality": "冷靜睿智", "world": "賽博龐克廢土", "goal": "修復城市最後的淨水系統"}, "gold_output": {"scenes": {"scene_1": {"
7  {"input": {"name": "圓堂", "personality": "永不放棄", "world": "現實足球場", "goal": "成為世界第一足球選手"}, "gold_output": {"scenes": {"scene_1": {"des
8  {"input": {"name": "李小龍", "personality": "看重友情且有義氣", "world": "東方武俠世界", "goal": "成為武林盟主"}, "gold_output": {"scenes": {"scene_1": {"
9  {"input": {"name": "陳日", "personality": "勇敢", "world": "西方奇幻世界", "goal": "拯救被魔王囚禁的公主"}, "gold_output": {"scenes": {"scene_1": {"des

```

For objectively evaluate to measure text similarity. I choose to use metrics such as ROUGE, BLEU, and BERTScore to measure text similarity. And use structural pass rate to check the system can correct to follow the created story can fullfil my script structure and logical norms.

Here is the result:



Structural pass rate: 66.67%

Average BERTScore: 0.6677

Average ROUGE-1: 0.0000

Average ROUGE-L: 0.0000

Average BLEU: 0.0478

Structure pass rate: 66.67%

For a system requiring strict adherence to JSON format, a 66% pass rate means that one out of every three generation attempts will cause the program to crash or the story to be interrupted. It may be my system to generate the whole 20 story scenes and options at once. LLM will easily cause some problems by creating too many things. Maybe I can reduce the number of story scenes or adjust my prompt to make the model fix the problem.

Average BERTScore: 0.6677

This indicates that the content generated by the model is semantically related to the input. Even if the output doesn't match, the model at least knows it's writing about football, martial arts, or adventure stories; it's not entirely off-topic.

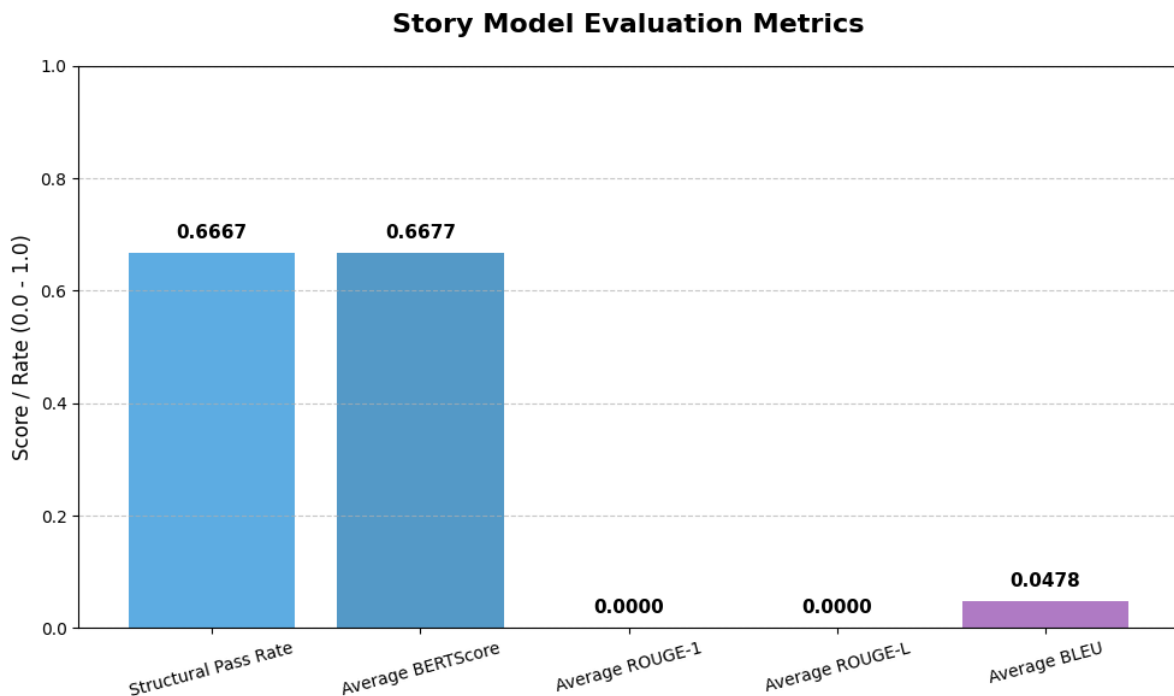
ROUGE:

For my search, the rouge-score default is designed for English. It is designed to segment words by spaces. When processing Chinese, if you don't manually "characterize" the sentence or use jieba for word segmentation, it will treat the entire long string of Chinese characters as a single, very large character, causing the match to fail.

BLEU (0.0478): This reflects that the sentence structure and grammatical conventions of the model-generated sentences differ significantly from the standard answer. But follow the BERTScore it means that while the sentences generated by the model are semantically correct.

For my manual check, the story follows the input and it is readable.

Follow the data to add some rules to require the system must follow my script structure and logical norms and other adjustments. Success increases the structural compliance rate. But still can not make the ROUGE scale work properly. It needs to be fixed or find other scales to replace it.



Structure Compliance Rate: 77.78%

Average BERTScore: 0.6659

Average ROUGE-1: 0.0000

Average ROUGE-L: 0.0000

Average BLEU: 0.0381

Human Evaluation:

Automatic metrics are not enough for testing user experience, so I invite real users to experience my system. Ask them to grade the system on a 5-point Likert scale across various criteria, such as usability, engagement, and clarity. I find my family to do this research. From their feedback I find that my font and window size should be larger. So I do the function to let them choose size they want. The user interface is clean and very simple to use. The story is simple to understand and the option clearly affects the story.

Dimension	ID	Evaluation Metrics	Score (1-5)
1. Usability	U1	The font size is appropriate and easy to read without strain.	3
	U2	The operation logic is simple (e.g., clicking options) and not confusing.	5
	U3	I can clearly tell when it is my turn to act versus when the system is processing.	5
	U4	The system response is stable, making me feel in control and secure.	5
2. Engagement	E1	The story content is warm and relatable, making me want to continue playing.	4
	E2	The characters have distinct personalities that I can empathize with.	4
	E3	I feel that my choices truly influence the story, giving me a sense of achievement.	4
	E4	The overall pace of the game is relaxed and pleasant, without causing pressure.	5
3. Clarity	C1	The story background and goals (e.g., saving the princess) are very clear.	5
	C2	The text for options is specific; I can predict what will happen after clicking.	5
	C3	The language used is colloquial and free of difficult technical jargon.	5
	C4	The visual color scheme is gentle, and transitions between scenes are logical.	5

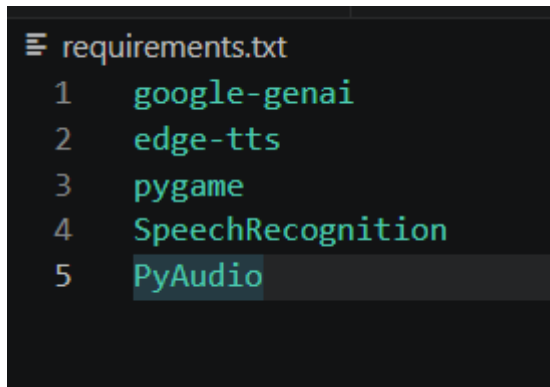
Requirement

Python 3.12 (64bit): The newest version of Python, such as Python 3.14, is too new, some libraries are not supported. Python 3.12 is my develop version, so my program can run successfully in this version.

Geminin API: For create the story

VPN: Because Gemini API is not supported in the Hong Kong area.

requirements.txt: Help for installing all the library

A screenshot of a code editor with a dark background. The file name 'requirements.txt' is shown in the top left corner. Below it, five lines of code are listed, each preceded by a line number from 1 to 5. The code lists the following libraries: 'google-genai', 'edge-tts', 'pygame', 'SpeechRecognition', and 'PyAudio'. The 'PyAudio' line is highlighted with a light blue background.

```
1 google-genai
2 edge-tts
3 pygame
4 SpeechRecognition
5 PyAudio
```

Possible function and future problem

Music and background: Let the user enjoy the story more.

VPN problem: Now the Gemini API can not be called from the Hong Kong area. So need VPN to call and apply. But Google already say that it will open the Hong Kong area limitation. Now we can already use the website version of Gemini. So this problem should be fix very soon in the future. In that time did not need to use VPN anymore.